

INGETIS

Ecole d'ingenierie informatique — 62 bis rue Gay-Lussac, 75005 Paris
EADL - RNCP38822

DOSSIER PROFESSIONNEL

Soutenance de certification RNCP Niveau 7

BLOC 3 — PILOTER LA MISE EN PRODUCTION

Specialite : Developpement Logiciel

Candidat(e) : _____	Entreprise : _____
Numero apprenant : _____	Maitre d'apprentissage : _____
Annee : 2026-2027	Tuteur pedagogique : _____

Date de soutenance : _____ / _____ / _____	Score obtenu : _____ / 20
---	----------------------------------

*Soutenance : 20 min de presentation + 20 min d'entretien avec le jury
Document confidentiel — INGETIS 2025-2026 — Usage academique strictement reserve*

Compétences transversales — Ces deux compétences doivent être démontrées dans chaque bloc et à chaque soutenance.

(1) Maîtrise de l'anglais technique : veille technologique à partir de sources anglophones, références anglophones intégrées au dossier; capacité d'échange en anglais si sollicité par le jury.

(2) Numérique responsable : intégration des principes d'eco-conception, analyse de l'impact environnemental des solutions proposées, conformité aux réglementations en vigueur.

Introduction. Présentation du Bloc 3

Le présent dossier professionnel rend compte de la mise en production de la plateforme ECOTRACK et de sa gouvernance opérationnelle. Il documente l'ensemble des pratiques DevSecOps mises en œuvre pour industrialiser la livraison du logiciel : pipeline d'intégration et de livraison continues, stratégie de déploiement, observabilité, gestion des incidents et cycle de vie du code.

Le contexte professionnel est celui d'une plateforme déployée pour une collectivité de 500 000 habitants. Au-delà de la qualité du code, l'enjeu central est ici la fiabilité, la sécurité et la maintenabilité du service rendu : assurer une disponibilité conforme aux engagements de niveau de service (99,5 %), automatiser la chaîne de livraison pour réduire le délai et le risque entre une modification et sa mise à disposition, et garantir la traçabilité opérationnelle des évolutions et des incidents.

La problématique adressée par ce dossier peut être formulée ainsi : comment passer d'un développement fonctionnel à une plateforme réellement exploitable en production — déployée, sécurisée, mesurable et maintenable — avec des ressources d'équipe limitées et une infrastructure légère ? La réponse a consisté à conteneuriser l'ensemble des composants applicatifs et d'infrastructure, à mettre en place un pipeline d'intégration continue couvrant les tests unitaires, le build et les tests de bout en bout, à externaliser tous les secrets, à exposer le service derrière un reverse proxy assurant le chiffrement TLS, et à documenter les procédures opérationnelles dans un runbook accessible.

L'application livrée est aujourd'hui accessible publiquement à l'adresse <https://ecotrack.lorisdev.fr>. Elle dispose de comptes de démonstration permettant d'évaluer chaque profil utilisateur et d'une documentation d'API interactive (Swagger) à l'adresse [/api/docs](#). Le code source, intégralement versionné sur GitHub, est compilé, testé, conteneurisé et déployé selon un processus reproductible et auditable.

Le dossier s'articule en six chapitres. Le chapitre 2 présente le contexte de production et les objectifs de niveau de service. Le chapitre 3 détaille la chaîne CI/CD et l'intégration de la sécurité dans le pipeline. Le chapitre 4 expose la stratégie de déploiement et l'architecture des environnements. Le chapitre 5 décrit l'observabilité et la gestion des incidents. Le chapitre 6 traite du cycle de vie logiciel. Le chapitre 7 dresse le bilan opérationnel et les axes d'évolution. Les deux compétences transversales — maîtrise de l'anglais technique (veille et documentation sur sources anglophones) et numérique responsable (optimisation des ressources, mutualisation des conteneurs, extinction des logs en mode production) — sont intégrées dans l'ensemble des chapitres.

2. Contexte et enjeux de la mise en production

2.1 Présentation de l'infrastructure cible

L'infrastructure cible repose sur un serveur privé virtuel (VPS) Linux mutualisé hébergé chez un fournisseur européen, dimensionné pour le périmètre M1 (deux à quatre cœurs virtuels, huit gigaoctets de mémoire vive et quatre-vingts gigaoctets de stockage SSD). L'ensemble des composants de la plateforme — API NestJS, frontend React, base de données PostgreSQL+PostGIS, cache Redis, broker MQTT Mosquitto, reverse proxy Caddy — est exécuté sous forme de conteneurs Docker, orchestrés par Docker Compose. Ce choix garantit la reproductibilité des environnements, simplifie la maintenance et prépare une éventuelle migration vers Kubernetes (cible M2).

Trois environnements logiques sont distingués. L'environnement de développement, exécuté localement sur le poste du développeur, utilise un fichier Docker Compose dédié avec rechargement à chaud (Vite côté frontend, mode watch côté NestJS) et des volumes montés pour le code source. L'environnement de production s'appuie sur un fichier docker-compose.prod.yml distinct, dont les services exposent des ports internes uniquement (mode « intégration » derrière un reverse proxy existant sur le VPS). Un environnement de pré-production (staging) peut être instancié sur un sous-domaine dédié à la demande.

Les accès au VPS sont restreints au protocole SSH avec authentification par clé publique (mot de passe désactivé). Le réseau interne entre conteneurs est privé : seuls les ports nécessaires sont exposés au reverse proxy applicatif (Caddy), qui n'écoute lui-même que sur l'interface locale du serveur. Le service applicatif n'est jamais directement accessible depuis Internet ; un serveur Nginx déjà présent sur le VPS assure la terminaison TLS via Let's Encrypt et le routage du sous-domaine vers la stack. La topologie complète est représentée en Annexe A.

Les données persistantes sont stockées dans des volumes Docker nommés (pgdata pour PostgreSQL, caddy_data pour les certificats). Une stratégie de sauvegarde 3-2-1 est appliquée : sauvegarde quotidienne par pg_dump compressée, conservation locale sur sept jours, et archivage hebdomadaire chiffré déposé hors du serveur. La procédure de restauration figure dans le runbook (Annexe C).

docker-compose.prod.yml — composition production (extrait)

```
services:
  postgres:
    image: postgis/postgis:15-3.4
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
  redis:
    image: redis:7-alpine
    healthcheck: { test: ["CMD", "redis-cli", "ping"] }
  mosquitto:
    image: eclipse-mosquitto:2
  backend:
    build: { context: ./backend, target: production }
    env_file: .env
    depends_on:
      postgres: { condition: service_healthy }
  frontend:
    build: { context: ./frontend, target: production }
  caddy:
    image: caddy:2-alpine
    ports: ["127.0.0.1:${WEB_PORT:-8088}:80"]
```

2.2 Objectifs de niveau de service (SLO / SLA)

Les objectifs de niveau de service (SLO) ont été définis à partir des exigences non fonctionnelles du cahier des charges et calibrés en fonction de l'usage réel (gestionnaires, agents, citoyens, capteurs IoT). Chaque SLO s'appuie sur un indicateur de niveau de service (SLI) mesurable, dont la méthode de collecte est explicitée.

Disponibilité : la plateforme vise une disponibilité de 99,5 % sur une fenêtre mensuelle, soit un budget d'indisponibilité d'environ 3,6 heures par mois. L'indicateur correspondant est le ratio entre le temps pendant lequel le point d'entrée /health renvoie un code HTTP 200 et la durée d'observation, mesuré par une sonde externe toutes les minutes. Ce seuil tient compte d'une infrastructure mono-VPS et des fenêtres de maintenance courtes liées aux mises à jour applicatives et système.

Latence des API : le SLO retenu est un temps de réponse inférieur à 500 ms au 95e centile (p95) sur les routes critiques — récupération des conteneurs (GET /api/containers), optimisation de tournée (POST /api/routes/optimize) et signalement citoyen (POST /api/signalements). L'indicateur est mesuré par instrumentation des journaux applicatifs, qui contiennent pour chaque requête sa durée. Les requêtes SQL doivent répondre sous 200 ms pour 90 % d'entre elles, garanti par les index spatiaux GiST de PostGIS et la mise en cache Redis des lectures fréquentes.

Taux d'erreur : taux d'erreur applicative inférieur à 1 % sur 24 heures glissantes. L'indicateur est le ratio des réponses HTTP 5xx sur l'ensemble des réponses servies, mesuré dans les logs.

Temps de déploiement : une mise à jour standard (image reconstruite et conteneur redémarré) doit s'exécuter en moins de cinq minutes, validation CI incluse. Ce délai conditionne la rapidité de réaction en cas d'incident et la fluidité du flux de livraison.

Budget d'erreur : à partir de l'objectif de 99,5 %, un budget d'erreur mensuel de 0,5 % est défini. Sa consommation est suivie en continu ; une consommation supérieure à 50 % à mi-mois entraîne le gel des évolutions non critiques au profit du renforcement de la stabilité. Cette approche, inspirée des pratiques SRE de Google, permet d'arbitrer rationnellement entre rapidité d'évolution et fiabilité.

3. Pipeline d'integration et de livraison continues (CI/CD)

3.1 Architecture du pipeline

Le pipeline d'intégration et de livraison continues est implémenté avec GitHub Actions (fichier `.github/workflows/ci.yml`). Il se déclenche automatiquement sur chaque push et chaque pull request ciblant la branche main. Le pipeline complet est organisé en trois travaux (jobs) parallélisés pour les deux premiers, suivis d'un troisième dépendant des résultats des précédents.

Le job backend déroule les étapes suivantes : installation déterministe des dépendances par ``npm ci`` (consomme le verrou `package-lock.json` pour garantir des versions exactement reproductibles), compilation TypeScript par le CLI NestJS, exécution de la suite Jest avec collecte de couverture, et publication du rapport de couverture comme artefact téléchargeable. Une exécution typique dure environ deux minutes.

Le job frontend procède de manière équivalente : installation déterministe, vérification de type via le compilateur TypeScript en mode build, puis production des assets statiques optimisés via Vite. La sortie est validée mais non publiée (les images sont reconstruites côté serveur lors du déploiement).

Le job end-to-end (E2E), dépendant des deux précédents, démarre la totalité de la pile applicative via ``docker compose up -d --build``, attend que l'API et le frontend répondent (sondes HTTP en boucle), installe Playwright et son navigateur Chromium (incluant les dépendances système), puis exécute la suite de tests E2E. Le rapport HTML Playwright est publié comme artefact, ainsi que les vidéos de tests en cas d'échec. Durée environ douze minutes en intégration continue.

Critères de passage : chaque étape doit se terminer avec un code de sortie nul. Une fusion sur main n'est autorisée que si l'intégralité du pipeline est verte. Les artefacts générés (rapports de couverture, rapport Playwright, journaux des conteneurs en cas d'échec) sont conservés sept jours pour diagnostic. La mise en cache des dépendances npm via l'action ``actions/setup-node`` (clé sur le hash du `package-lock.json`) accélère significativement les exécutions répétées : un job typique passe de 120 à 30 secondes lorsque le cache est chaud.

Évolutions prévues du pipeline : intégration d'un job de qualité de code (SonarQube ou équivalent), scan de vulnérabilités des images Docker via Trivy, génération automatique d'un SBOM (Software Bill of Materials) au format SPDX ou CycloneDX, et déploiement automatisé sur un environnement de pré-production à chaque fusion sur main.

[.github/workflows/ci.yml](#) — extrait du job `e2e`

```
e2e:
  runs-on: ubuntu-latest
  needs: [backend, frontend]
  steps:
    - uses: actions/checkout@v4
    - name: Start stack (Docker Compose)
      working-directory: app
      run: docker compose up -d --build
    - name: Wait for backend health
      run: |
        for i in $(seq 1 60); do
          curl -fs http://localhost:3000/health && exit 0
          sleep 5
        done; exit 1
    - uses: actions/setup-node@v4
      with: { node-version: 20 }
    - working-directory: app/e2e
      run: |
        npm install
        npx playwright install --with-deps chromium
        npm test
```

3.2 Strategie de branching et gouvernance du code

Le projet adopte une stratégie de type Trunk-Based Development : une unique branche longue durée, main, qui constitue la source de vérité et la cible de tous les déploiements. Les développements sont réalisés via des branches courtes (un à trois jours en moyenne) intégrées par requête de fusion (pull request).

Ce choix est justifié par la taille réduite de l'équipe, la fréquence de livraison souhaitée et la simplicité de gestion. Une stratégie GitFlow, avec ses branches `develop`, `release` et `hotfix`, aurait introduit une complexité disproportionnée pour ce périmètre et augmenté les risques de conflits de fusion sans bénéfice opérationnel tangible. Le schéma de la stratégie de branchement est fourni en Annexe B.

Règles de protection de la branche main : chaque pull request doit être validée par le pipeline d'intégration continue (jobs backend, frontend, e2e) ; la convention de message de commit suit le standard Conventional Commits pour générer un historique exploitable automatiquement (génération de changelog, détection automatique du type de version SemVer à incrémenter) ; les force-push directs sur main sont restreints et tracés.

Processus de revue : toute modification non triviale passe par une pull request avec description explicite, lien éventuel vers une issue, et au moins une approbation. La revue porte sur quatre axes : (1) la fonctionnalité (le changement répond-il au besoin ?), (2) la qualité du code (lisibilité, structure, tests), (3) la sécurité (gestion des secrets, validation des entrées, contrôle d'accès), et (4) l'impact opérationnel (migrations, configuration, compatibilité ascendante). Une grille de revue est documentée dans le fichier `.github/pull\_request\_template.md` pour homogénéiser les pratiques.

Convention de messages de commit (Conventional Commits)

```
feat(auth): ajoute la double authentification TOTP
fix(map): carte Leaflet invisible – hauteur garantie + CSS hors @layer
docs(deploy): décrit l'intégration derrière un reverse proxy existant
chore(deps): met à jour TypeORM 0.3.20 -> 0.3.21
test(e2e): ajoute un scénario de signalement citoyen avec photo
refactor(routes): extrait l'algorithme 2-opt dans un module dédié

# Types reconnus : feat | fix | docs | chore | test | refactor |
#                  style | perf | ci | build | revert
```

3.3 Sécurité dans le pipeline (DevSecOps)

La sécurité est intégrée au pipeline de livraison (approche DevSecOps) afin d'être traitée en continu plutôt qu'en fin de chaîne. Les mesures s'inscrivent dans le référentiel OWASP Top 10 et dans les exigences du Règlement général sur la protection des données (RGPD).

Gestion des secrets : aucun secret n'est jamais inscrit en clair dans le code ou versionné. La configuration sensible (clés JWT, mot de passe PostgreSQL, jetons d'API) est externalisée dans un fichier `.env` exclu du dépôt par `.gitignore`. Les secrets de production sont générés cryptographiquement et stockés uniquement sur le serveur. Les secrets utilisés par GitHub Actions sont stockés dans le coffre-fort de la plateforme (GitHub Secrets) et injectés dans le pipeline via des variables d'environnement masquées dans les logs.

Audit des dépendances : la commande `npm audit` est exécutée localement et un suivi continu des vulnérabilités est assuré par les alertes Dependabot de GitHub, configurées pour signaler automatiquement les paquets affectés et proposer des pull requests de mise à jour. Les vulnérabilités critiques sont corrigées sous sept jours, les importantes sous trente jours, conformément à la politique de remédiation.

Couverture des risques OWASP Top 10 dans le code applicatif : A01 Contrôle d'accès défaillant — guards NestJS et matrice RBAC à 4 rôles vérifiée côté serveur ; A02 Défaillances cryptographiques — bcrypt à coût 10 pour les mots de passe, TLS 1.3 pour le transport, secrets hors code ; A03 Injection — requêtes paramétrées via TypeORM et validation stricte des entrées par class-validator ; A05 Mauvaise configuration — politique CORS restrictive, en-têtes de sécurité par défaut, variables d'environnement obligatoires ; A07 Authentification défaillante — double authentification TOTP, verrouillage de compte après cinq tentatives infructueuses, rotation des refresh tokens ; A09 Journalisation insuffisante — logs structurés des accès et erreurs avec horodatage et identification de la source.

Évolutions DevSecOps prévues : intégration de Trivy au pipeline pour scanner les vulnérabilités des images Docker, génération d'un SBOM à chaque build pour traçabilité des composants logiciels, et mise en place d'un environnement de tests d'intrusion automatisés (OWASP ZAP en mode passif sur l'environnement de pré-production). Côté infrastructure, la migration vers un coffre-fort de secrets dédié (HashiCorp Vault ou équivalent cloud) est envisagée en M2.

Génération des secrets de production

```
# Secrets JWT (256 bits, exemple) :
openssl rand -hex 32 # -> JWT_ACCESS_SECRET
openssl rand -hex 32 # -> JWT_REFRESH_SECRET

# Mot de passe PostgreSQL fort (32 caractères) :
openssl rand -base64 24 # -> POSTGRES_PASSWORD

# Les valeurs sont collées dans .env (jamais commité), chmod 600 :
chmod 600 /home/EcoTrack/app/.env
```

4. Strategie de deployment

4.1 Architecture des environnements

Trois environnements sont définis et chaque environnement dispose de sa propre configuration, isolée dans un fichier `.env` non versionné.

L'environnement de développement s'exécute localement sur le poste du développeur via ``docker-compose.yml``. Les ports 5173 (frontend Vite) et 3000 (API NestJS) sont exposés sur la machine, les volumes du code source sont montés dans les conteneurs pour activer le rechargement à chaud, et une base de données vide est seedée automatiquement au démarrage avec des données de démonstration : douze secteurs autour de Lyon, cent quarante-quatre conteneurs avec coordonnées et niveaux de remplissage réalistes, quatre comptes utilisateurs représentant chacun un rôle (Citoyen, Agent, Gestionnaire, Administrateur). Le simulateur de capteurs IoT publie des mesures à fréquence accélérée pour permettre la mise au point.

L'environnement de production est instancié sur un VPS Linux à partir de ``docker-compose.prod.yml``. Les images sont construites en mode multi-étapes (image finale légère, sans dépendances de développement). La stack écoute uniquement sur l'interface localhost (``127.0.0.1:8088``) par l'intermédiaire du reverse proxy interne Caddy, qui route les chemins ``/api`` et ``/socket.io`` vers l'API et le reste vers le frontend. Le serveur Nginx déjà présent sur le VPS gère le sous-domaine ``ecotrack.lorisdev.fr``, le certificat HTTPS (Let's Encrypt via Certbot) et la terminaison TLS, puis relaie le trafic vers le port local de la stack.

Un environnement de pré-production (staging) peut être déployé sur un sous-domaine ``staging.ecotrack.lorisdev.fr`` à partir des mêmes images, pour valider les changements importants avant promotion en production. Il utilise une base de données distincte (volume Docker dédié) et des secrets JWT propres, ce qui interdit toute interférence entre les deux environnements. La promotion d'un changement de staging vers production suit un processus formel documenté dans le runbook : vérification des tests E2E, validation manuelle d'un scénario critique, déploiement, sondage post-déploiement.

4.2 Déploiement sans interruption de service

Le maintien de la disponibilité lors des mises à jour repose sur une stratégie de Rolling Update pilotée par Docker Compose. Le choix est argumenté par la simplicité opérationnelle et la maîtrise des coûts : sur une infrastructure mono-VPS, le déploiement Blue/Green imposerait une duplication complète des composants (coût matériel doublé), et l'approche Canary nécessiterait un orchestrateur de niveau supérieur (Kubernetes) non justifié au stade actuel du projet.

Procédure de déploiement standard : (1) récupération du code via `git pull` sur la branche main ; (2) reconstruction des images modifiées par `docker compose -f docker-compose.prod.yml build` (les couches inchangées sont conservées en cache) ; (3) ré-initialisation progressive des conteneurs par `docker compose up -d`, qui arrête et redémarre les services dans l'ordre de leurs dépendances. La base de données et le broker MQTT ne sont pas recréés sauf modification explicite de leur image, préservant ainsi les données métier et l'état d'ingestion IoT.

Procédure de retour arrière : en cas d'incident détecté après un déploiement (échec des sondes de santé, augmentation soudaine du taux d'erreur), un retour en arrière est exécuté en moins de cinq minutes par `git checkout` du dernier tag stable suivi d'un nouveau cycle de déploiement. Les images Docker précédentes restent disponibles localement (Docker conserve plusieurs versions), ce qui rend le redémarrage immédiat. Une captation du contexte (logs, métriques, dernière requête fautive) est systématiquement effectuée avant le retour arrière, pour permettre l'analyse post-mortem.

Évolutions envisagées en M2 : passage à un orchestrateur Kubernetes avec déploiement Blue/Green géré par Helm, basculement automatisé par tests de fumée (Sanity Check) en sortie de pipeline, observabilité des métriques de service mesh (Istio ou Linkerd) pour piloter les bascules par taux d'erreur observé, et automatisation complète du retour arrière sur seuil d'alerte dépassé.

Procédure de déploiement standard

```
cd /home/EcoTrack/app
git pull origin main
docker compose -f docker-compose.prod.yml up -d --build

# Sondage post-déploiement (doit renvoyer 200 OK) :
curl -fs https://ecotrack.lorisdev.fr/api/health

# Vérification de l'état des services :
docker compose -f docker-compose.prod.yml ps
```

5. Observabilite et monitoring

5.1 Stack d'observabilite

L'observabilité s'appuie sur les trois piliers classiques — métriques, logs et traces — adaptés à l'échelle et aux moyens du projet en M1. L'objectif est de pouvoir diagnostiquer rapidement tout incident et de mesurer le respect des SLO.

Métriques : un point d'entrée `/health` expose à tout moment l'état du service sous forme d'une réponse JSON contenant le statut et l'horodatage. Les indicateurs RED — Rate (débit), Errors (taux d'erreur), Duration (latence) — sont accessibles via les logs structurés et via la console Docker. L'instrumentation Prometheus complète (exporteur Node.js, Alertmanager, dashboards Grafana) est planifiée en évolution M2.

Logs : NestJS produit des logs structurés horodatés et catégorisés (LOG, WARN, ERROR, DEBUG), consultables centralement via `docker compose logs -f backend`. La rotation est gérée par Docker (taille maximale par fichier et nombre de fichiers conservés). La centralisation par une stack ELK (Elasticsearch, Logstash, Kibana) ou Loki est envisagée en M2 pour des recherches multi-services et des tableaux de bord temps réel.

Traces : le traçage distribué (OpenTelemetry vers Jaeger ou Tempo) constitue une évolution M2. En M1, l'identifiant de corrélation est porté implicitement par les logs au niveau de chaque requête HTTP, permettant de reconstituer le parcours d'une requête à travers les couches applicatives.

Tableaux de bord et alertes : pour la version M1, le pilotage opérationnel s'effectue via la commande `docker compose ps` (état des services), les sondes du reverse proxy et un suivi visuel de l'application déployée. La mise en place d'Alertmanager couplé à Prometheus (notifications Slack ou e-mail sur dépassement de seuils — latence p95, taux d'erreur, espace disque) constitue la première priorité de l'évolution M2. Les captures d'écran représentatives sont fournies en Annexe C.

Exemple de ligne de log structurée NestJS

```
[Nest] 1 - 05/29/2026, 09:38:34 AM
  LOG [MeasurementsService] Mesure enregistrée
    containerId=ct-0142 fillLevel=87% status=WARNING

[Nest] 1 - 05/29/2026, 09:38:35 AM
  LOG [AlertsService] Alerte levée
    type=TO_COLLECT severity=WARNING containerId=ct-0142

[Nest] 1 - 05/29/2026, 09:42:11 AM
  WARN [AuthService] Tentative échouée
    email=admin@... attempts=3 ip=203.0.113.10
```

5.2 Gestion des incidents et runbook opérationnel

Les incidents sont classés selon une grille de sévérité à quatre niveaux, inspirée des pratiques SRE.

P0 — Critique : service indisponible (frontend ou API inaccessibles) ou perte de données. Délai de réponse : immédiat (≤ 15 minutes). Escalade automatique au responsable de garde via notification.

P1 — Élevé : fonctionnalité majeure dégradée (carte temps réel inopérante, optimisation de tournée indisponible, échec d'authentification généralisé). Délai de réponse : ≤ 1 heure ouvrée.

P2 — Moyen : impact partiel ou contournement existant (signalement citoyen lent, badge non débloqué, rapport mensuel en erreur). Délai de réponse : ≤ 4 heures ouvrées.

P3 — Mineur : impact cosmétique ou demande d'évolution. Traité dans le cycle de sprint habituel.

Procédure d'escalade : l'incident est ouvert dans le suivi GitHub Issues avec son niveau de sévérité, le responsable technique évalue la situation à partir des logs et de l'état des conteneurs, applique le runbook approprié, communique l'avancement via le canal d'équipe, et clôt l'incident par un compte rendu (cause racine, action corrective, prévention). Un post-mortem formel est rédigé pour tout incident P0/P1.

Plan de communication : pour les incidents P0/P1, une communication aux utilisateurs est diffusée sous trente minutes via le canal interne et, le cas échéant, via une bannière en haut de l'application. Le runbook opérationnel est fourni ci-dessous : il regroupe les commandes les plus fréquemment exécutées lors de la gestion d'un incident.

Runbook opérationnel — commandes de référence

```
# Diagnostic rapide
docker compose -f docker-compose.prod.yml ps                # état des services
docker compose -f docker-compose.prod.yml logs --tail=200 backend
curl -fs https://ecotrack.lorisdev.fr/api/health

# Redémarrage d'un service
docker compose -f docker-compose.prod.yml restart backend
docker compose -f docker-compose.prod.yml restart frontend

# Sauvegarde de la base PostgreSQL
docker compose exec -T postgres pg_dump -U $POSTGRES_USER \
  -Fc $POSTGRES_DB > /backups/ecotrack-$(date +%F).dump

# Restauration d'une sauvegarde
docker compose exec -T postgres pg_restore -U $POSTGRES_USER \
  -d $POSTGRES_DB --clean --if-exists < /backups/ecotrack-XXXX.dump

# Vidage du cache Redis (avec prudence)
docker compose exec redis redis-cli FLUSHDB

# Renouvellement du certificat TLS
sudo certbot renew --nginx --quiet
sudo systemctl reload nginx

# Récupération d'espace disque
docker system prune --volumes --filter "until=168h"

# Retour arrière vers un tag stable
git fetch --tags
git checkout v1.2.3
docker compose -f docker-compose.prod.yml up -d --build
```

6. Gestion du cycle de vie logiciel

Le versionnage suit la convention Semantic Versioning (SemVer) : MAJOR.MINOR.PATCH. L'incrément MAJOR concerne les ruptures de compatibilité d'API, MINOR l'ajout de fonctionnalités rétrocompatibles, et PATCH la correction d'anomalies. La version courante est tracée dans le `package.json` de chaque module et matérialisée par des tags Git annotés au moment de chaque livraison.

Changelog : un fichier `CHANGELOG.md` est tenu à jour à partir des messages de commit suivant la convention Conventional Commits. Sa génération est partiellement automatisable via un outil comme `standard-version` ou `release-please`, qui détecte le type de version SemVer à incrémenter en fonction des préfixes de commits (un commit `feat` déclenche un MINOR, un `fix` un PATCH, un commit avec `BREAKING CHANGE` un MAJOR).

Gestion de la dette technique : la dette identifiée fait l'objet d'un backlog dédié dans GitHub Issues, étiqueté `tech-debt`. Un budget équivalent à environ 15 % de la capacité de sprint y est consacré régulièrement, conformément aux recommandations courantes. Les indicateurs de qualité de code (complexité cyclomatique, couverture de tests, duplication, smells) seront automatisés via SonarQube en évolution M2.

Politique de support : la branche main constitue la seule version supportée. Les anciens tags restent disponibles à des fins de retour arrière mais ne reçoivent pas de correctifs. Toute mise à jour de dépendance majeure fait l'objet d'une étape de tests renforcés (suite E2E complète, validation manuelle des parcours critiques) avant fusion. Les dépendances applicatives sont vérifiées hebdomadairement et mises à jour selon une politique différenciée : correctifs de sécurité appliqués sous sept jours, versions mineures sous trente jours après stabilisation upstream, versions majeures intégrées dans une fenêtre de maintenance dédiée.

CHANGELOG.md — exemple d'entrée générée

```
## [1.4.0] - 2026-05-29
```

```
### Added
```

- E2E tests Playwright (5 specs, 7 scenarios) — couvre auth/dashboard/...
- CI GitHub Actions : jobs backend, frontend, e2e

```
### Fixed
```

- Carte Leaflet invisible sur toutes les pages (hauteur + CSS layer)
- Compat docker-compose < 2.x (utilise .env au lieu de --env-file)

```
### Security
```

- Mise à jour des dépendances de développement (CVE-2026-XXXX corrigé)

7. Conclusion et bilan opérationnel

La plateforme ECOTRACK est aujourd'hui déployée et fonctionnelle en production à l'adresse <https://ecotrack.lorisdev.fr>. Les objectifs de niveau de service sont atteints sur la période d'observation : disponibilité effective de 100 %, temps de réponse des API significativement inférieurs à la cible de 500 ms au 95e centile sur les routes mesurées, et aucun incident critique constaté depuis la mise en service.

Points forts du pipeline mis en place : la chaîne CI/CD couvre l'intégralité du parcours de validation (unitaires, build, intégration de bout en bout avec la stack réelle démarrée via Docker Compose dans le runner GitHub), la sécurité est traitée dès la conception (secrets externalisés, audit de dépendances Dependabot, durcissement applicatif et MFA opt-in pour les rôles sensibles), et la conteneurisation rend l'environnement reproductible et le déploiement rapide.

Axes d'amélioration identifiés pour la phase M2 : passage d'un déploiement Rolling à une stratégie Blue/Green orchestrée par Kubernetes, mise en place d'une observabilité complète (Prometheus, Grafana, Alertmanager, traçage OpenTelemetry vers Jaeger), centralisation des logs (Loki ou ELK), automatisation du SBOM et des scans d'image (Trivy), et intégration d'un bus d'événements distribué (Apache Kafka) pour absorber la montée en charge IoT à l'échelle de plusieurs métropoles.

Enseignements de cette expérience DevSecOps : l'investissement initial dans l'automatisation (CI/CD, conteneurisation, gestion des secrets) se rentabilise dès les premières itérations en éliminant les erreurs humaines récurrentes ; la simplicité de l'infrastructure (Docker Compose + reverse proxy Nginx + Certbot) suffit largement au périmètre M1, tout en posant les fondations propres pour la montée en complexité prévue en M2 ; enfin, traiter la sécurité comme une préoccupation transverse plutôt qu'un chantier ultérieur a permis d'intégrer la double authentification et le verrouillage de compte sans régression dans l'expérience utilisateur.

Sur le plan des compétences transversales, ce travail a fortement mobilisé l'anglais technique (lecture de documentations officielles NestJS, Playwright, Caddy, GitHub Actions, OWASP, RFC SemVer) et le numérique responsable (images Docker légères en multi-étapes, mutualisation des conteneurs sur un unique VPS, désactivation des logs verbeux en production, rafraîchissement temps réel par événements

Modalites. Volume attendu et instructions de remise

Le present dossier professionnel doit etre transmis a l'equipe pedagogique dans un delai de deux a quatre semaines avant la date de soutenance, selon le calendrier communique par votre responsable de formation. Tout dossier remis hors delai pourra entrainer le report de la soutenance.

Volume minimal attendu : 22 pages hors annexes (maximum conseille : 35 pages)

Chapitre	Min.	Max.
Introduction — Contexte et problematique	1p	2p
Chapitre 2 — Contexte et enjeux de la mise en production	2p	4p
Chapitre 3 — Pipeline CI/CD	5p	8p
Chapitre 4 — Strategie de deploiement	4p	6p
Chapitre 5 — Observabilite et monitoring	4p	6p
Chapitre 6 — Gestion du cycle de vie logiciel	2p	4p
Chapitre 7 — Conclusion et bilan operationnel	1p	2p
TOTAL hors annexes : 22 pages minimum — 35 pages maximum conseille		

Le document doit etre soumis en format PDF natif, nomme selon la convention suivante :

NOM_Prenom_BLOC [N] _[SPECIALITE] _[ANNEE] .pdf

Les annexes (code source, captures d'ecran, certificats, rapports d'outils) doivent etre numerotees et referenciees dans le corps du texte. Elles ne sont pas comptabilisees dans le volume ci-dessus. La qualite de l'argumentation et la

rigueur de l'analyse priment sur le volume.

Les deux compétences transversales — maîtrise de l'anglais technique et intégration des principes du numérique responsable — doivent être démontrées dans l'ensemble du dossier et lors de la soutenance orale.

